

Análise de Desempenho de Algoritmos de Ordenação sob Estresse

EMANUEL RAUTER, Cartão: 00578247

GEANCARLO KOZENIESKI, Cartão: 00264414

TIAGO ELIAS STEINKE, Cartão: 00171884

Este trabalho apresenta uma avaliação experimental de algoritmos de ordenação sob condições controladas de estresse de CPU e memória. São analisados HeapSort, MergeSort e QuickSort por meio de um planejamento fatorial completo que considera diferentes distribuições de vetores de entrada, tipos de estresse e níveis de carga. Os experimentos foram automatizados e executados de forma isolada, garantindo reprodutibilidade por meio de seeds fixas e execução sequencial, totalizando 840 execuções.

As métricas avaliadas incluem tempo de execução, consumo de memória, consumo de energia e custo algorítmico (comparações e trocas). Adicionalmente, cargas sintéticas foram aplicadas para simular contenção de recursos de CPU, memória e ambos simultaneamente.

Os resultados indicam que o QuickSort apresenta melhor desempenho na maioria dos cenários, enquanto o HeapSort é mais eficiente em vetores com alta repetição de elementos. O MergeSort demonstra comportamento estável, porém com maior consumo de memória. A análise estatística (ANOVA) mostra que o tipo de vetor e a carga de execução são os fatores mais influentes no desempenho, enquanto os algoritmos reagem de forma semelhante ao estresse aplicado.

1 Introdução

Algoritmos de ordenação eficientes utilizam diferentes estratégias para reduzir o tempo de processamento, sendo frequentemente analisados sob uma perspectiva teórica baseada em complexidade assintótica. No entanto, esse enfoque nem sempre reflete o comportamento observado em cenários reais de execução. Neste trabalho, é realizado um estudo experimental em ambiente controlado para avaliar o impacto de estresse em CPU e memória RAM sobre o desempenho de diferentes algoritmos de ordenação, considerando distintos conjuntos de dados.

Os experimentos seguem um planejamento fatorial completo, no qual todas as combinações entre algoritmos, tipos de entrada e níveis de estresse são avaliadas. Essa abordagem permite analisar de forma sistemática as interações entre os algoritmos e as condições de execução, fornecendo evidências empíricas sobre seu comportamento em ambientes computacionais reais.

2 Fundamentação Teórica

2.1 Algoritmos de Ordenação

Os algoritmos de ordenação são utilizados em diversos problemas computacionais e constituem uma das bases da Ciência da Computação. Embora diversos algoritmos possuam complexidade assintótica semelhante, diferenças no uso de estruturas auxiliares, padrões de acesso à memória e número de operações podem impactar significativamente seu tempo de execução.

Neste trabalho foram avaliados três algoritmos de ordenação baseados em comparação: HeapSort, MergeSort e QuickSort. Todos possuem complexidade assintótica média de $O(n \log n)$, porém apresentam comportamentos distintos durante a execução.

Quicksort: algoritmo de divisão e conquista que utiliza um pivô para particionamento do vetor. Após a partição, elementos menores que o pivô são posicionados à esquerda e maiores à direita, permitindo ordenação recursiva.

Quando o pivô gera partições equilibradas, o algoritmo mantém complexidade $O(n \log n)$. Entretanto, partições desfavoráveis podem degradar o desempenho, levando ao pior caso de $O(n^2)$.

Neste trabalho, foi utilizada uma estratégia de seleção de pivô com o objetivo de reduzir a ocorrência de partições degeneradas. O Quicksort também opera in-place, dispensando memória auxiliar e apresentando boa localidade de referência.

Mergesort: algoritmo de divisão e conquista que divide recursivamente o vetor em duas metades até que cada subvetor possua um único elemento. Em seguida, realiza a intercalação ordenada desses subvetores até reconstruir o vetor final ordenado.

Apresenta complexidade $O(n \log n)$ independentemente da distribuição dos dados de entrada. Entretanto, requer um vetor auxiliar, com custo adicional de memória $O(n)$.

Apesar desse custo, o algoritmo realiza acessos sequenciais durante a intercalação, o que favorece a localidade de referência e o aproveitamento das hierarquias de cache.

Heapsort: organiza os elementos em uma heap binária máxima. Em seguida, o maior elemento é removido da raiz e colocado na posição final do vetor, com restauração da propriedade de heap após cada remoção.

Apresenta complexidade $O(n \log n)$ em todos os casos e utiliza apenas memória auxiliar constante $O(1)$. No entanto, devido às reorganizações da estrutura de heap, realiza acessos não sequenciais à memória, resultando em menor localidade de referência.

2.2 Estresse de Recursos Computacionais

O desempenho computacional de algoritmos é fortemente influenciado pela disponibilidade de recursos de hardware compartilhados. Em sistemas multicore, há competição por unidades de execução, memória principal e hierarquias de cache, o que pode gerar interferência entre processos simultâneos.

Para avaliar esse impacto, foi implementado um mecanismo de geração de carga controlada durante a execução dos algoritmos de ordenação. Foram considerados quatro modos de operação:

- **None:** execução sem estresse adicional;
- **CPU:** aplicação de carga apenas sobre o processador;
- **RAM:** aplicação de carga apenas sobre a memória principal;
- **Both:** aplicação simultânea de carga sobre CPU e memória.

Além do tipo de estresse, foram considerados dois níveis de intensidade, correspondentes a 50% e 100% da carga máxima configurada.

Estresse de CPU

O estresse de CPU é produzido por um conjunto de threads independentes que executam continuamente operações aritméticas de ponto flutuante durante toda a execução do algoritmo de ordenação.

O número de threads é definido automaticamente com base na quantidade de núcleos lógicos disponíveis, obtida por meio de `std::thread::hardware_concurrency()`, sendo escalado pelo fator de carga definido para o experimento. Dessa forma, o nível de intensidade controla diretamente a quantidade de threads responsáveis pela geração de carga computacional.

Cada thread executa continuamente operações de multiplicação, divisão, soma e subtração até o término do algoritmo avaliado, mantendo ocupadas as unidades de execução do processador sem realizar operações de entrada e saída.

Estresse de Memória RAM

O estresse de memória principal é realizado por uma thread dedicada que aloca um buffer cujo tamanho é proporcional ao nível de carga do experimento.

O tamanho do buffer é calculado como uma fração de 32 GB, permitindo aumentar progressivamente o uso de memória conforme a intensidade do estresse. Após a alocação, o buffer é completamente inicializado antes do início da medição, evitando interferência no tempo de execução.

Durante toda a execução do algoritmo de ordenação, a thread realiza acessos pseudoaleatórios de leitura e escrita sobre o buffer utilizando um gerador baseado no algoritmo XorShift64. Essa estratégia reduz a previsibilidade dos acessos e dificulta o uso de mecanismos de prefetch e cache, aumentando a pressão sobre a largura de banda da memória principal.

É importante destacar que o limite máximo de estresse corresponde a 32 GB de memória alocada, desconsiderando aproximadamente 5 GB utilizados pelo sistema operacional. Dessa forma, cargas superiores a cerca de 27 GB podem levar à utilização de memória virtual, impactando significativamente o desempenho dos benchmarks.

Estresse Combinado

No cenário de estresse combinado (Both), as cargas de CPU e memória são executadas simultaneamente. Inicialmente, a thread responsável pelo estresse de memória realiza a alocação e inicialização completa do buffer. Somente após essa etapa são iniciadas as threads de CPU, garantindo que a medição do algoritmo de ordenação ocorra apenas quando todas as cargas concorrentes já estejam ativas.

Ao término da execução do algoritmo, todas as threads de estresse são sincronizadas e encerradas de forma controlada, assegurando condições experimentais reproduzíveis e consistentes entre execuções.

3 Metodologia Experimental

3.1 Ambiente Experimental

Ambiente: Laptop AORUS 15P XD.

Processador: 11th Gen Intel i7-11800H (8 cores, 16 threads).

Memória RAM: 32 GB DDR4 3200 MHz Dual-channel (2x16 GB).

Armazenamento: SSD 1 TB M.2 2280 PCIe 4.0 NVMe.

Sistema Operacional: Ubuntu 24.04.3.

Kernel: 6.17.0-23-generic.

A escolha do sistema operacional Ubuntu deve-se ao menor consumo de recursos em comparação ao Windows 11, permitindo execuções de benchmark mais estáveis e com menor interferência de processos em segundo plano.

3.2 Conjunto de Dados

Para avaliar o desempenho dos algoritmos de ordenação, foram utilizados vetores contendo 10^7 elementos. Esse tamanho foi escolhido como compromisso entre tempo de execução viável e volume suficiente para reduzir variações do sistema operacional e do hardware.

Foram considerados oito tipos de vetores:

- (1) Ordenado: elementos em ordem crescente.
- (2) Reverso: elementos em ordem decrescente.
- (3) Aleatório: valores distribuídos aleatoriamente.
- (4) Repetidos: vetor com um único valor repetido.
- (5) Repetidos (1 a 5): valores restritos ao intervalo [1,5].
- (6) Zig-Zag: alternância entre valores máximos e mínimos.
- (7) Ordenado com ruído: 1% dos elementos finais randomizados.
- (8) Ordenado com trocas: 1% de trocas aleatórias no vetor.

Todos os conjuntos de dados são gerados automaticamente antes de cada repetição experimental utilizando seeds fixas, garantindo reprodutibilidade.

3.3 Planejamento Experimental

O benchmark segue um planejamento fatorial completo, considerando combinações entre algoritmos, tipos de vetor, tipos de estresse e intensidade de carga. Cada configuração é executada 5 vezes.

Antes de cada execução, o ambiente é configurado com os parâmetros do experimento. Caso haja estresse, threads adicionais são criadas para gerar carga concorrente. No estresse de memória, a alocação do buffer ocorre antes do início da medição; somente após sua conclusão as threads de CPU são iniciadas.

Após a sincronização das threads de estresse, inicia-se a medição do tempo de execução e a execução do algoritmo. Ao final, as threads são encerradas e o vetor é verificado para garantir correteude.

Os resultados são armazenados em arquivos CSV para análise posterior.

O total de execuções é dado por 8 tipos de vetores, 7 configurações de estresse e 3 algoritmos, totalizando 168 combinações. Com 5 repetições, obtém-se 840 execuções.

3.4 Automação dos Experimentos

Os experimentos foram automatizados via script Bash responsável por compilação, geração de dados e execução dos cenários.

O script utiliza compilação com otimização -O2, cria diretórios de resultados e gera datasets a cada repetição com seeds fixas. Em seguida, percorre todos os arquivos e executa os algoritmos sob os diferentes cenários de estresse.

O cenário sem estresse é executado uma única vez, enquanto CPU, RAM e combinação são avaliados nos níveis de carga definidos.

3.5 Isolamento Experimental

A execução dos experimentos é orquestrada por script Bash executado com systemd-inhibit, prevenindo suspensão ou interrupções do sistema durante os testes.

Antes de cada execução, os resultados anteriores são removidos e o acesso ao subsistema Intel RAPL é habilitado para coleta de energia. As execuções ocorrem de forma sequencial, garantindo isolamento entre experimentos.

3.6 Métricas de Desempenho

Foram coletadas as seguintes métricas:

- Tempo de execução (ms);
- Consumo máximo de memória (KB);
- Consumo médio de memória (KB);
- Consumo de energia (J);
- Número de comparações;
- Número de trocas.

Como o estresse introduz consumo adicional de recursos, métricas de energia e memória são analisadas apenas no cenário sem estresse. Nos demais casos, o tempo de execução é a métrica principal.

3.7 Análise Estatística

Foram construídos intervalos de confiança de 95% para as médias, permitindo avaliar a significância das diferenças observadas.

Também foram calculadas métricas derivadas:

- *Slowdown*: razão entre tempo com estresse e tempo sem estresse;
- Eficiência energética: operações por joule;
- Eficiência temporal: operações por segundo.

Por fim, foi realizada ANOVA considerando algoritmo, tipo de vetor, estresse, intensidade e suas interações. Foram reportados estatística F, valor-p e eta quadrado parcial (η^2), com nível de significância de 5% ($\alpha = 0,05$).

4 Resultados

Esta seção apresenta os resultados obtidos a partir dos experimentos descritos na Seção 3. Inicialmente são analisadas as métricas individuais de desempenho (tempo, memória e energia). Em seguida, avalia-se o impacto do estresse computacional sobre cada algoritmo, realiza-se uma comparação geral entre os métodos de ordenação e, por fim, apresenta-se uma análise estatística dos fatores que influenciam o desempenho.

4.1 Tempo de Execução

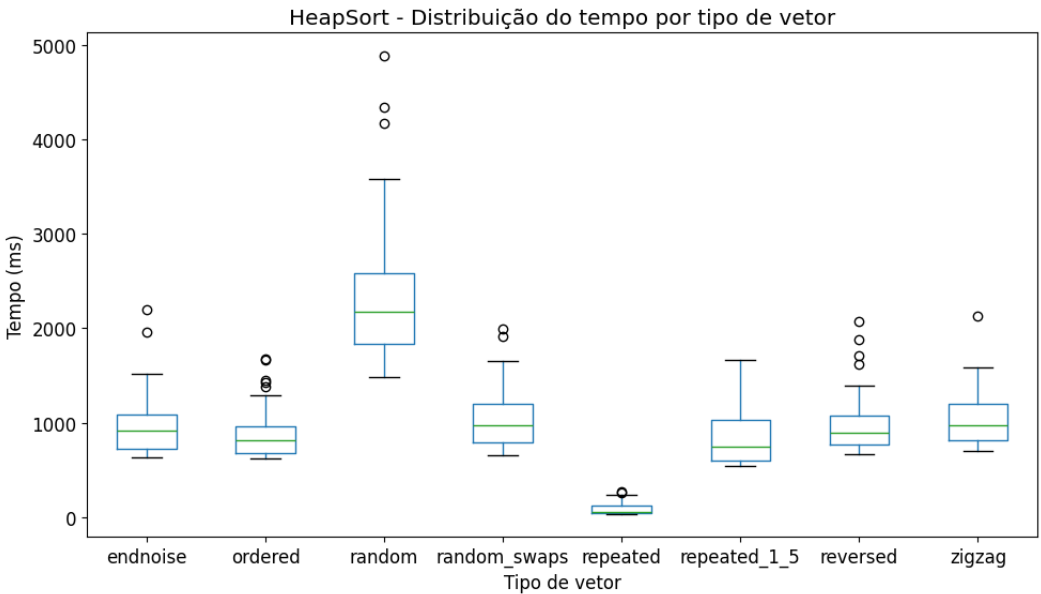


Fig. 1. Tempo de execução do HeapSort por vetor

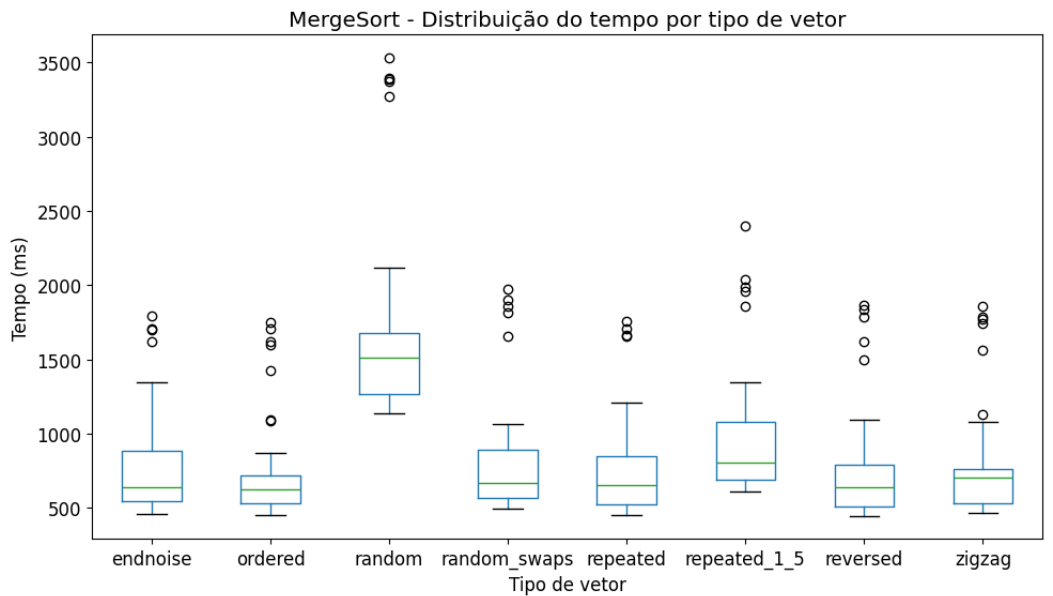


Fig. 2. Tempo de execução do MergeSort por vetor

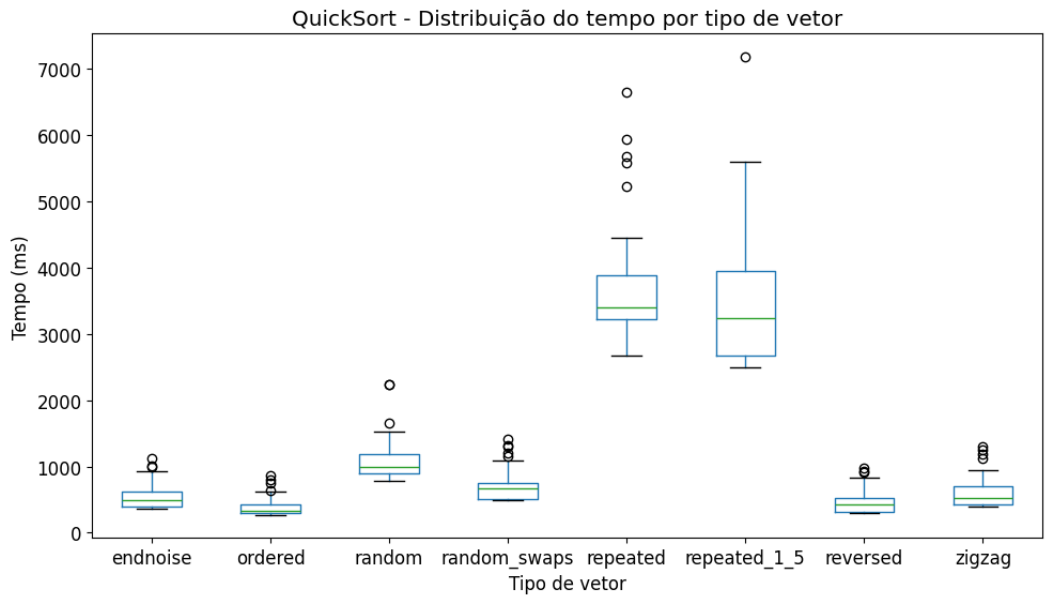


Fig. 3. Tempo de execução do QuickSort por vetor

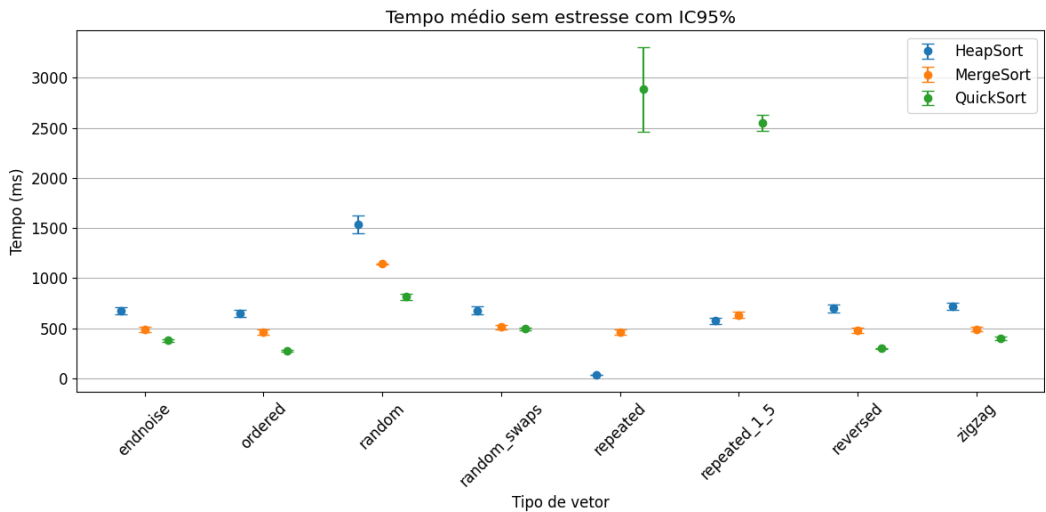


Fig. 4. Tempo de todos sorts por vetor com IC 95

O Quicksort apresentou o menor tempo de execução na maioria dos cenários, com melhor desempenho no vetor aleatório, onde HeapSort e MergeSort apresentaram maior custo computacional. Entretanto, observou-se degradação significativa em vetores com grande número de elementos repetidos, caracterizando seu pior caso prático neste conjunto de testes.

O HeapSort apresentou melhor desempenho em cenários com alta repetição de valores, atingindo tempos inferiores a 100 ms em vetores com um único valor dominante, indicando baixa sensibilidade à distribuição dos dados.

O MergeSort apresentou comportamento altamente consistente, mantendo-se como a segunda melhor opção na maioria dos cenários sem estresse. Seu melhor desempenho ocorreu em vetores com perturbações aleatórias, onde se aproximou do desempenho do Quicksort.

Para HeapSort e MergeSort, vetores com valores repetidos foram os casos mais favoráveis, enquanto vetores aleatórios foram os mais custosos. Para o Quicksort, vetores ordenados foram os mais favoráveis e vetores com valores repetidos os mais desfavoráveis.

4.2 Consumo de Memória

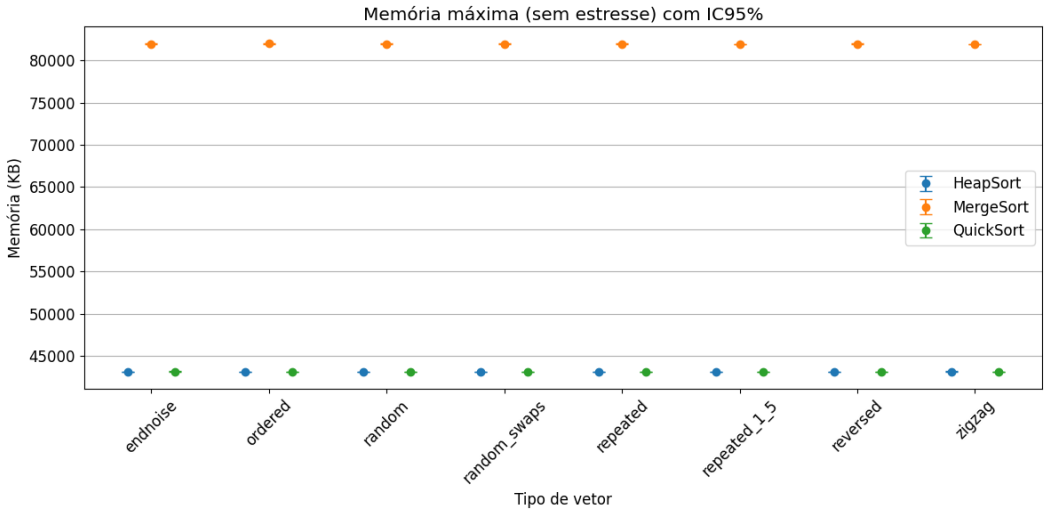


Fig. 5. Uso de memória máxima RAM por vetor com IC 95

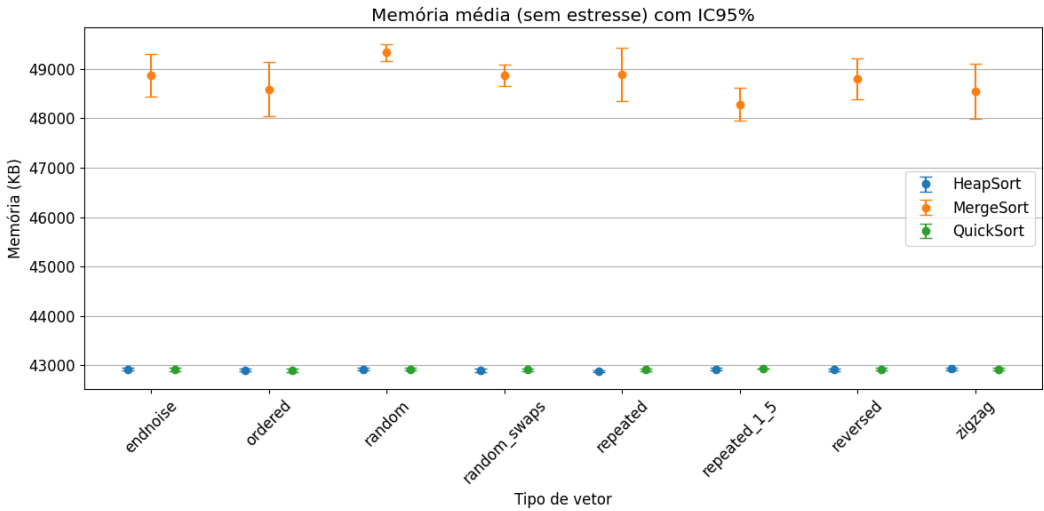


Fig. 6. Uso de memória máxima RAM por vetor com IC 95

HeapSort e Quicksort apresentaram uso de memória estável, com consumo máximo aproximado de 43 MB e média próxima de 42,9 MB, indicando que o tamanho do vetor é o principal determinante do consumo.

O MergeSort apresentou consumo aproximadamente duas vezes maior, devido ao uso de vetor auxiliar proporcional ao tamanho da entrada. Além disso, apresentou maior variação no uso médio de memória, refletindo sua estrutura interna de intercalação.

4.3 Consumo de Energia

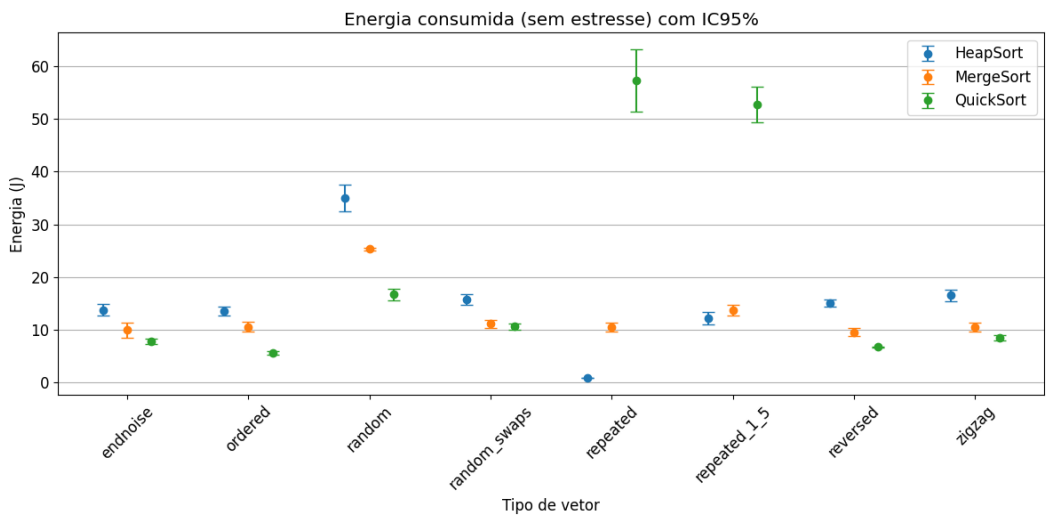


Fig. 7. Uso de energia por vetor com IC 95

Observou-se forte correlação entre consumo de energia e tempo de execução. Em todos os casos, execuções mais longas resultaram em maior consumo energético, indicando relação aproximadamente linear entre essas métricas.

4.4 Impacto do Estresse no Desempenho

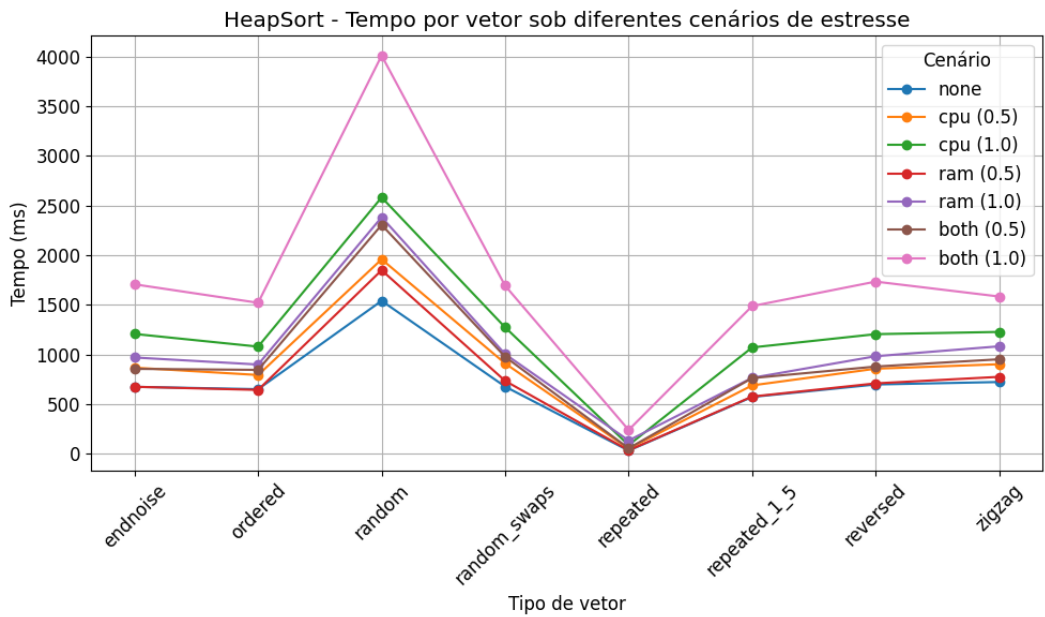


Fig. 8. Influência do estresse no tempo do Heap

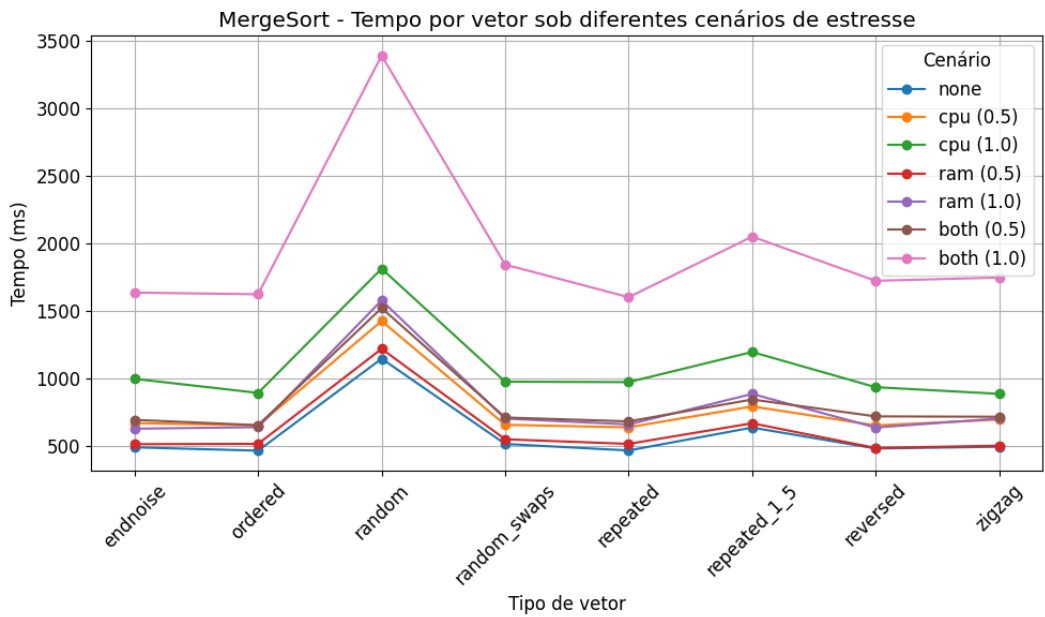


Fig. 9. Influência do estresse no tempo do Merge

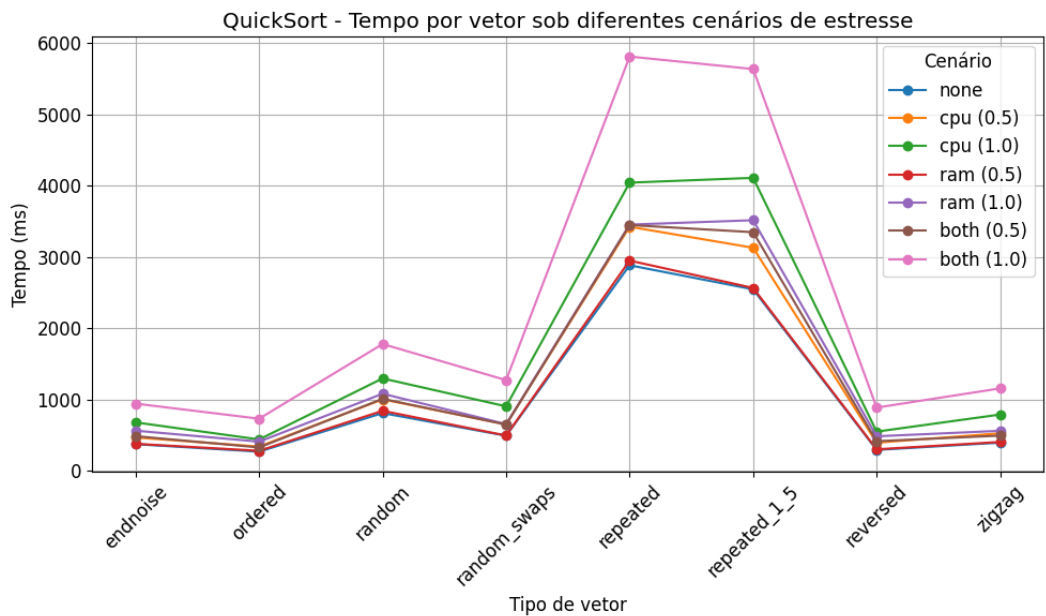


Fig. 10. Influência do estresse no tempo do Quick

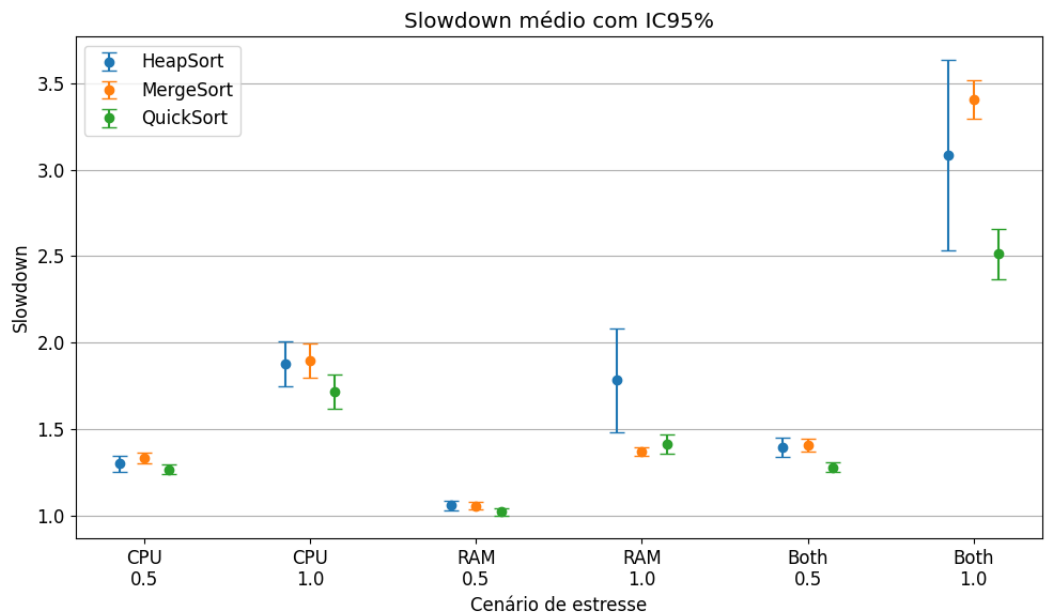


Fig. 11. Slowdown de todo algoritmo por nível de estresse

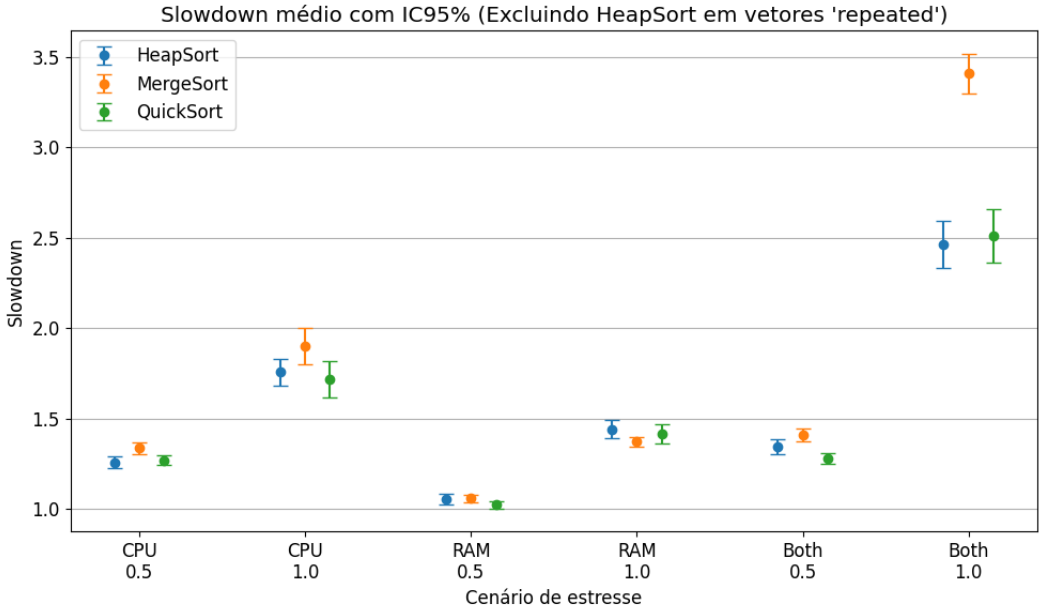


Fig. 12. Slowdown com anomalia removido

A análise dos resultados evidencia padrões consistentes entre os algoritmos. O estresse de CPU apresentou aumento quase linear do slowdown conforme o nível de carga, enquanto o estresse de memória em 50% apresentou impacto reduzido em relação ao cenário sem estresse.

O impacto do estresse de memória torna-se mais significativo quando a alocação se aproxima ou ultrapassa a capacidade física da RAM, levando ao uso de memória virtual e aumento substancial do custo computacional. Esse comportamento é observado principalmente no nível de 100% de carga.

O estresse combinado (CPU + RAM) produziu as maiores degradações de desempenho, com efeitos não necessariamente aditivos. Isso sugere interação entre contenção de processamento e largura de banda de memória, amplificando o impacto total.

Analisando individualmente, o MergeSort mostrou maior sensibilidade ao estresse de CPU, com maiores variações em cenários de carga combinada. Para o HeapSort, foram omitidos gráficos envolvendo o vetor *repeated* em algumas análises visuais devido à baixa variabilidade temporal desse caso, o que poderia distorcer a escala dos resultados.

4.5 Comparação e Ranking dos Algoritmos

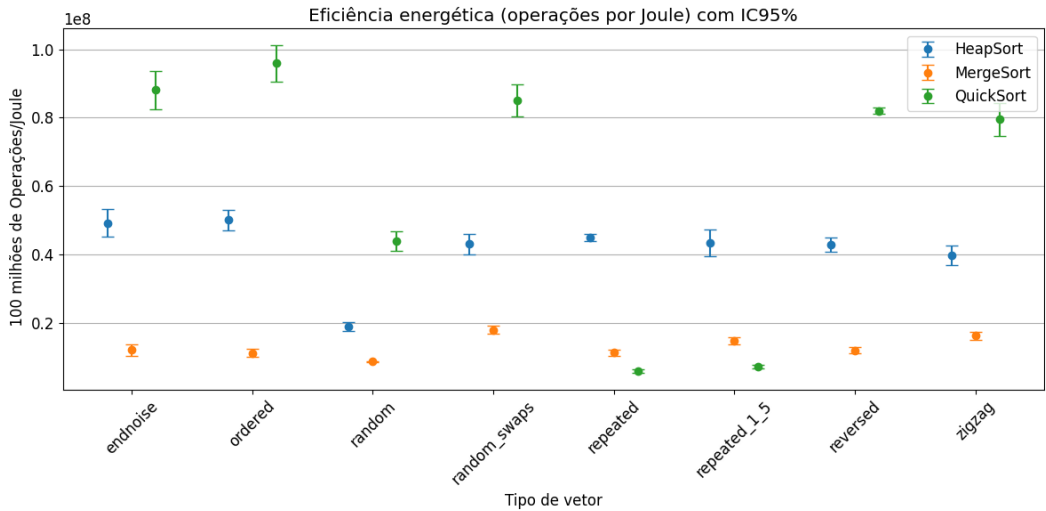


Fig. 13. Eficiência energética

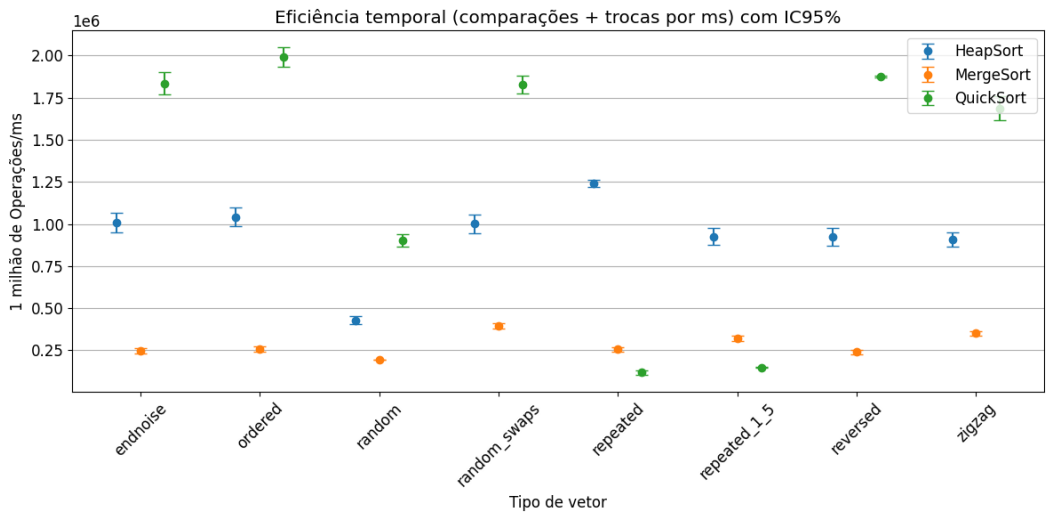


Fig. 14. Eficiência temporal

O MergeSort apresentou menor eficiência em operações por milissegundo e por joule, o que é esperado devido à ausência de operações in-place e ao uso de vetor auxiliar. No entanto, em cenários onde o Quicksort apresenta degradação, o MergeSort torna-se mais competitivo.

O HeapSort apresentou desempenho intermediário tanto em eficiência temporal quanto energética. O Quicksort foi o mais eficiente na maioria dos cenários, exceto nos casos de degeneração.

Fator	F	p-valor	η^2 parcial
C(algorithm)	165.59	< 0.001	0.291
C(vector_type)	245.06	< 0.001	0.680
C(stress)	141.63	< 0.001	0.345
C(carga)	452.30	< 0.001	0.529
C(algorithm):C(stress)	1.84	0.089	0.014
C(vector_type):C(algorithm)	319.59	< 0.001	0.847

Table 1. Resultados da ANOVA para o tempo de execução.

4.6 Análise de Variância (ANOVA)

A análise de variância indica que todos os fatores principais (algoritmo, tipo de vetor, estresse e carga) possuem efeito estatisticamente significativo sobre o tempo de execução.

A interação entre algoritmo e tipo de estresse não foi significativa ($p > 0.05$), indicando comportamento semelhante entre algoritmos sob diferentes tipos de estresse. Em contraste, a interação entre algoritmo e tipo de vetor apresentou forte significância estatística, sendo o fator com maior tamanho de efeito.

O coeficiente η^2 indica que aproximadamente 84,5% da variabilidade do tempo de execução é explicada pela interação entre algoritmo e tipo de vetor, enquanto a interação com o estresse explica menos de 1% da variação observada.

5 Discussão

Os resultados obtidos demonstram que o desempenho de algoritmos de ordenação depende simultaneamente das características dos dados de entrada e das condições de execução do sistema. Embora todos os algoritmos apresentem complexidade assintótica $O(n \log n)$ no caso médio, diferenças em padrões de acesso à memória, número de comparações e movimentações de elementos resultam em comportamentos significativamente distintos na prática.

O QuickSort apresentou o menor tempo de execução na maioria dos cenários avaliados. Seu bom desempenho pode ser atribuído à elevada localidade espacial durante o particionamento, ao reduzido uso de memória auxiliar e ao baixo fator constante de sua implementação. Entretanto, sua vantagem diminui em vetores com grande quantidade de elementos repetidos, cenário no qual o HeapSort apresenta melhor desempenho.

O HeapSort apresentou comportamento consistente em todas as distribuições de entrada, sem degradações significativas. Seu desempenho superior em vetores com muitos elementos repetidos sugere menor sensibilidade a duplicatas quando comparado ao QuickSort. Por outro lado, seu maior número de comparações e acessos não sequenciais à memória contribui para pior desempenho na maioria dos demais cenários.

O MergeSort apresentou desempenho intermediário e estável, mantendo comportamento consistente entre diferentes distribuições de entrada. Apesar do menor número de comparações e de sua boa localidade de acesso, o uso de vetor auxiliar implica maior consumo de memória.

A análise dos cenários de estresse indica que tanto a contenção de CPU quanto de memória aumentam o tempo de execução, sendo o estresse combinado (CPU + RAM) o mais impactante. O slowdown observado cresce aproximadamente de forma proporcional à intensidade da carga, indicando consistência do mecanismo de estresse aplicado.

Os resultados da ANOVA reforçam essas observações. O tipo de estresse apresenta efeito significativo no tempo de execução, porém sem interação estatisticamente significativa com o algoritmo, indicando comportamento semelhante entre os métodos sob estresse. Em contraste, a interação entre

algoritmo e tipo de vetor apresenta o maior tamanho de efeito, evidenciando que as características da entrada são o fator mais determinante.

Observa-se ainda forte correlação entre tempo de execução e consumo de energia, indicando que algoritmos mais rápidos tendem a consumir menos energia nas condições avaliadas. Isso sugere que otimizações de tempo podem também melhorar eficiência energética.

De forma geral, não existe um algoritmo universalmente superior. O QuickSort é mais eficiente na maioria dos casos, o HeapSort se destaca em entradas com alta repetição, e o MergeSort apresenta desempenho estável, porém com maior custo de memória. Esses resultados reforçam que a escolha do algoritmo deve considerar não apenas complexidade teórica, mas também características dos dados e do ambiente de execução.

6 Ameaças à Validade

Todo estudo experimental está sujeito a limitações que podem influenciar os resultados obtidos. Nesta seção são discutidas as principais ameaças à validade deste trabalho e as estratégias adotadas para reduzi-las.

Uma primeira ameaça está relacionada às características do hardware utilizado. Os experimentos foram conduzidos em uma única plataforma computacional, de modo que diferenças de arquitetura, quantidade de memória, hierarquia de caches ou número de núcleos podem alterar o desempenho absoluto dos algoritmos. Entretanto, como todos os algoritmos foram executados sob exatamente as mesmas condições de hardware e software, as comparações realizadas permanecem válidas dentro do ambiente experimental considerado.

Outra limitação refere-se ao mecanismo de geração de estresse. Embora as cargas de CPU e memória tenham sido projetadas para produzir contenção controlada de recursos, elas não reproduzem necessariamente o comportamento de aplicações reais executadas simultaneamente. Ainda assim, o objetivo do trabalho é comparar o comportamento relativo dos algoritmos sob diferentes níveis de contenção, e não modelar uma carga de trabalho específica.

A variabilidade inerente ao sistema operacional também pode influenciar as medições, devido ao escalonamento de processos, interrupções, serviços em segundo plano e outros eventos externos. Para minimizar esses efeitos, cada configuração experimental foi executada cinco vezes, sendo posteriormente utilizadas medidas estatísticas como médias, desvios padrão, intervalos de confiança e análise de variância (ANOVA).

Outra possível ameaça decorre da seleção dos conjuntos de dados. Embora tenham sido utilizados oito padrões distintos de vetores, eles não representam todas as distribuições possíveis encontradas em aplicações reais. Ainda assim, foram escolhidos cenários amplamente empregados na literatura e capazes de representar diferentes graus de ordenação, repetição de elementos e padrões de acesso à memória.

Por fim, a avaliação concentrou-se em três algoritmos clássicos de ordenação (HeapSort, MergeSort e QuickSort) e em implementações específicas desenvolvidas em C++. Diferentes implementações, otimizações do compilador ou variantes desses algoritmos podem produzir resultados distintos. Dessa forma, as conclusões deste trabalho devem ser interpretadas como válidas para as implementações e condições experimentais avaliadas.

Apesar dessas limitações, o planejamento fatorial completo, a automação dos experimentos, a utilização de uma seed fixa para geração dos conjuntos de dados e a aplicação de métodos estatísticos conferem robustez às comparações realizadas e aumentam a reprodutibilidade dos resultados obtidos.

7 Conclusão

Este trabalho apresentou uma avaliação experimental do desempenho dos algoritmos HeapSort, MergeSort e QuickSort sob diferentes padrões de entrada e condições de contenção de recursos computacionais. Para isso, foi desenvolvido um ambiente automatizado de benchmark capaz de gerar conjuntos de dados, aplicar níveis controlados de estresse sobre CPU e memória e coletar métricas de tempo de execução, consumo de memória, consumo de energia e custo algorítmico.

Os resultados demonstraram que o QuickSort apresentou o melhor desempenho na maioria dos cenários avaliados, enquanto o HeapSort destacou-se em vetores compostos por elementos repetidos. O MergeSort apresentou desempenho intermediário e comportamento consistente, embora com maior consumo de memória em função da utilização de um vetor auxiliar.

A análise dos cenários de estresse mostrou que a contenção de recursos aumenta significativamente o tempo de execução dos algoritmos, sendo o estresse combinado sobre CPU e memória o cenário de maior impacto. Entretanto, a análise estatística indicou que os três algoritmos respondem de maneira semelhante aos diferentes níveis de estresse, enquanto as características dos vetores de entrada exercem influência significativamente maior sobre o desempenho relativo de cada algoritmo.

Os resultados da ANOVA corroboram essa conclusão ao identificar que o tipo de vetor, o algoritmo, o tipo de estresse e a intensidade da carga possuem influência estatisticamente significativa sobre o tempo de execução. Além disso, a interação entre algoritmo e tipo de vetor apresentou o maior tamanho de efeito, evidenciando que as propriedades dos dados de entrada são o fator que mais diferencia o comportamento dos algoritmos avaliados.

Por fim, a utilização de um planejamento fatorial completo, da automação integral dos experimentos e da aplicação de métodos estatísticos permitiu realizar uma comparação abrangente e reprodutível entre os algoritmos estudados. Dessa forma, os resultados obtidos fornecem uma caracterização experimental consistente do comportamento do HeapSort, MergeSort e QuickSort em diferentes condições de execução, servindo como referência para a escolha do algoritmo mais adequado conforme as características da entrada e do ambiente computacional.

References

- [1] GeeksforGeeks. Merge Sort Algorithm. <https://www.geeksforgeeks.org/dsa/merge-sort/>.
- [2] GeeksforGeeks. Heap Sort Algorithm. <https://www.geeksforgeeks.org/dsa/heap-sort/>.
- [3] GeeksforGeeks. Quick Sort Algorithm. <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>.